Chapter 8: Loops

An `if` statement checks a condition once and picks a branch. A **loop** checks a condition, runs a block, and checks the condition again, for as long as it holds. (1) Everything about how a condition is evaluated, and how a block runs once selected, already exists from the last chapter. A loop just asks the same question more than once.

```
i = 0;
while (i < 3) {
    print(i);
    i = i + 1;
}
```

This prints `0`, `1`, `2`. Each pass through the body checks `i < 3` again before deciding whether to run it a fourth time.

Breaking Changes

Earlier use	Change
A variable, or any identifier, named `while`, `break`, or `continue`	Rename it; all three names are now keywords.

Nothing about `if`, blocks, or the `(value, status)` pair changes. A loop is built entirely on top of what conditionals already established.

Tracing a Counter

Before writing any new evaluator code, trace `i = 0; while (i < 3) { ... }` by hand:

Check	`i` before	Condition	Runs body?	`i` after
1st	0	`0 < 3` = true	yes	1
2nd	1	`1 < 3` = true	yes	2
3rd	2	`2 < 3` = true	yes	3
4th	3	`3 < 3` = false	no	3

The condition is checked one more time than the body runs. That last check, the one that finally comes back false, is not an edge case to handle separately. It is the ordinary way every loop ends. The same table with an initial value of `3` instead of `0` has only one row: the body never runs at all.

Condition, Body, and Update

A loop has three moving parts, and losing track of the difference between them is the most common way to write one that never ends:

- **Condition**: `i < 3`. Checked before every iteration, including the first.
- **Body**: `print(i)`. What runs while the condition holds.
- **Update**: `i = i + 1`. What changes between one check of the condition and the next.

The update is not a separate kind of statement. It is an ordinary assignment that happens to live inside the body, in a position where it runs before the body ends and the condition is checked again. Move it, forget it, or write it in a way that never actually changes the value the condition tests, and the condition keeps evaluating to the same thing forever. `while (i < 3) { print(i); }` with no update at all prints `0` until something else stops the program.

Vertex's interpreter does not catch this. It only ever runs a program; it never looks the body over first to ask whether it could run forever. That is a limitation of this particular interpreter, not a law about loops in general: a condition that tests a variable the body never reassigns is exactly the kind of pattern a static checker could flag before the program ever runs. What is impossible is a single procedure that could decide, for every program written in a Turing-complete language, whether that program halts. No such procedure exists for any language expressive enough to simulate arbitrary computation; this is the halting problem. (2) Some deliberately restricted languages give up that expressiveness and can guarantee termination in exchange. This chapter's point is narrower than either case: whether any particular loop terminates depends on what its update actually does, not on its syntax, and nothing in this interpreter checks that for you.

Accumulating Values

The same three parts build up a running total:

```
n = 5;
sum = 0;
k = 1;
while (k <= n) {
    sum = sum + k;
    k = k + 1;
}
```

`sum` ends at `15`: `1 + 2 + 3 + 4 + 5`. `k` is doing two jobs at once here, as the value being added and as the counter the condition tests, which is common but not required.

One way to be sure this loop computes the right thing, beyond just running it and checking the final answer, is to state what stays true every time the condition is checked: at that point, `sum` always equals the sum of the integers from `1` up to `k - 1`. Check it against the trace from earlier: before the first check, `k` is `1` and `sum` is `0`, the sum of nothing, so it holds from the start. Each pass through the body adds `k` to `sum` and then increments `k`, which is exactly what keeps that statement true for the next check, including the last one, where `k` is `6` and `sum` is `15`, the sum of `1` through `5`. A claim like this, checked before the loop starts and preserved by every iteration, is a **loop invariant**. (3) It is a way to reason about whether a loop is correct independent of how many times it happens to run.

Strings accumulate the same way, through repeated concatenation instead of addition:

```
letters = "";
j = 0;
while (j < 3) {
    letters = letters + "x";
    j = j + 1;
}
```

`letters` ends at `"xxx"`.

Parsing a While Statement

```
while_statement ::= "while" "(" expression ")" block
```

A loop body is a block: the same mandatory braces as an `if` branch, reusing the same `parse_block` this chapter did not need to change at all.

```
def parse_while_statement(tokens):
    # while_statement ::= "while" "(" expression ")" block
    # A loop body is a block, the same requirement as an if branch: no bare
    # statement, and empty or semicolon-only is a legal body that does
    # nothing each time the condition holds.
    tokens = require(tokens, "while", "Expected 'while'")
    tokens = require(tokens, "(", "Expected '(' after 'while'")
    condition, tokens = parse_expression(tokens)
    tokens = require(tokens, ")", "Expected ')' after while condition")
    body, tokens = parse_block(tokens)
    return {"tag": "while", "condition": condition, "body": body}, tokens
```

The resulting tree has two parts: a condition expression and a body, which is an ordinary `statement_list` node, exactly like an `if` branch.

```
{
    "tag": "while",
    "condition": {"tag": "<", "left": {...}, "right": {...}},
    "body": {"tag": "statement_list", "statements": [...]},
}
```

`break` and `continue` need even less: each is just a keyword, with no condition, no argument, and no block of its own.

```
break_statement ::= "break"
continue_statement ::= "continue"
```

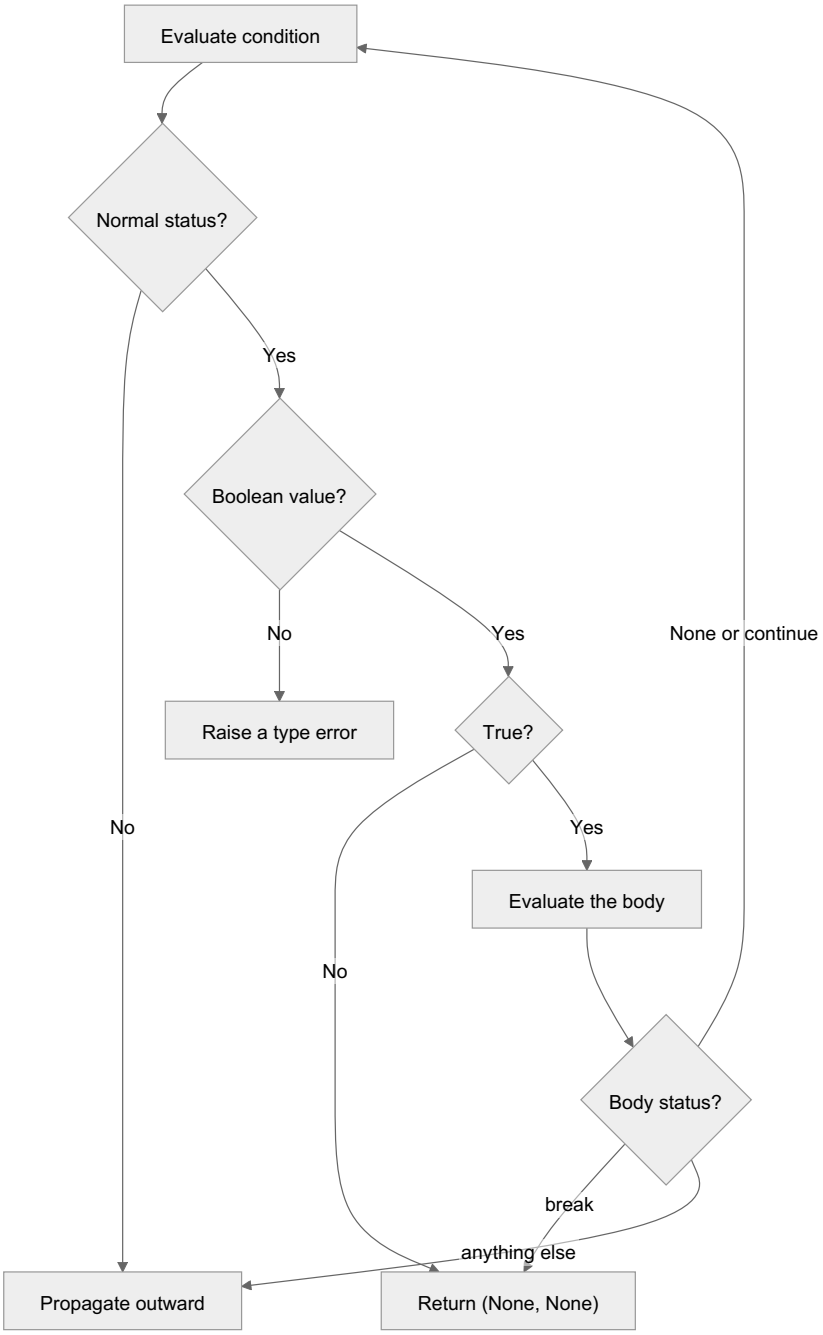
Evaluating a Loop

```
if ast["tag"] == "break":
    return None, "break"

if ast["tag"] == "continue":
    return None, "continue"

if ast["tag"] == "while":
    while True:
        condition, status = evaluate(ast["condition"], environment)
        if status is not None:
            return condition, status
        if type(condition) is not bool:
            raise TypeError("while condition must be a boolean expression")
        if not condition:
            return None, None
        value, status = evaluate(ast["body"], environment)
        if status == "break":
            return None, None
        if status is not None and status != "continue":
            # exit, a failed assertion, or anything else this loop does
            # not own: pass it outward without finishing the loop.
            return value, status
        # Normal completion and "continue" both return to the condition.
```

`break` and `continue` do not evaluate anything. They are leaves: reaching one simply reports a status, the same way `exit` reports `"exit"`. Nothing about that status says “loop” until something checks for it. A `while` node is the first thing in this interpreter that does.



Only two outcomes send this Python `while True:` loop back to the top to check the condition again: a normally completed body, and a body that ends in `"continue"`. Everything else ends it. A false condition ends it, and so does `break`, both by returning `(None, None)`, because from outside the loop, “the condition finally failed” and “something inside asked to stop” look identical: the loop is done, and nothing went wrong. `exit`, an assertion failure, or any status this `while` node does not recognize, whether it came from the condition or the body, ends it a third way: propagated outward immediately, without pretending the loop finished normally.

Break Finishes the Nearest Loop

```
found = 0;
search = 0;
while (true) {
    search = search + 1;
    if (search == 4) {
        break;
    };
    found = search;
}
```

`search` ends at `4`; `found` ends at `3`. `break` fires while `search` is `4`, before `found = search` runs for that pass, and the loop's own `while True:` returns `(None, None)` right there. The condition `true` is never checked a fifth time.

Continue Returns to the Condition

```
total = 0;
m = 0;
while (m < 5) {
    m = m + 1;
    if (m == 3) {
        continue;
    };
    total = total + m;
}
```

`total` ends at `12`: `1 + 2 + 4 + 5`, skipping `3`. The update `m = m + 1` runs before the `if`, so `continue` skips `total = total + m` but not the statement that advances `m`. `continue` performs no update of its own; it is an ordinary status, consumed by the nearest `while`, which responds to it by doing exactly what it does after an ordinary completed body: check the condition again. If the update lived after the `continue` instead of before it, this loop would never advance past `m = 3` and would run forever: the moment `m` reached `3`, `continue` would skip the update on every later pass too, and the condition `m < 5` would keep testing the same value.

Nested Loops

```
outer = 0;
combined = 0;
while (outer < 2) {
    outer = outer + 1;
    inner = 0;
    while (true) {
        inner = inner + 1;
        if (inner == 2) {
            break;
        };
        combined = combined + 1;
    };
    combined = combined + 10;
}
```

`combined` ends at `22`. Each pass through the outer loop resets `inner` to `0`, runs the inner loop until it breaks at `inner == 2` (adding `1` to `combined` once, on the iteration before that), and then adds `10` after the inner loop is done. The inner `break` only unwinds the inner `while` node's own Python loop; the outer `while` node never sees a `"break"` status at all, because the inner `while` already consumed it and returned `(None, None)` before the outer loop's body continued past `while (true) { ... }`.

Misuse Outside a Loop

Nothing in the grammar stops `break` or `continue` from appearing outside a `while` body. Nothing in the evaluator stops them either: `evaluate` for `"break"` and `"continue"` runs the same regardless of what encloses it. A stray `break` at the top level produces `(None, "break")`, and every ordinary node between it and the program's own `statement_list` propagates that pair unchanged, the same way an unrecognized status always has since the last chapter. It reaches the runner still unconsumed:

```
if status == "exit":
    # Keep process termination outside the recursive evaluator.
    sys.exit(value)
if status is not None:
    raise ValueError(f"Unhandled evaluation status: {status}")
```

`"break"` is not `"exit"`, so the runner's second check catches it and raises. This is not new code written for this chapter. It is the same fallback that would already fire for any status nothing in the program knew how to handle, because until now, only `exit` and a failed assertion ever produced one. Misuse of `break` or `continue` is diagnosed, just not by the parser, and not by a check written specifically for them.

Testing the Contract

`test_loops.py` covers zero iterations, one iteration, and repeated state changes; the boolean-condition requirement and rejection of unbraced bodies; `break` and `continue` skipping later body statements; `continue` reevaluating the condition exactly once per skip, checked with a mocked `input()` call count, not just by reading the result; nested loops, where an inner `break` leaves the outer loop's own remaining work intact; and `exit` and a failed assertion propagating out of a loop body instead of being consumed like `break`. Two tests run small inline Vertex programs through the runner as a subprocess with a bounded timeout: one checks that `break`/`continue` outside a loop is rejected, the other runs a handful of small `while` programs end to end. Vertex has no compilation step; these are interpreted from source directly, the same as everything else. Most of the other loop tests call the evaluator in-process with no timeout at all, so those two are a guard against the tests most likely to hang on an unbounded loop, not a guarantee covering the whole suite.

A Larger Example

`newton.v`, alongside the chapter's own example, computes square roots with Newton's method using nothing beyond what this chapter and the ones before it built: arithmetic, comparisons, `while`, `if`, and `assert`. The update step is the whole algorithm in one line:

```
guess = (guess + S / guess) / 2;
```

and the convergence check needs the language's own `if` to do something `abs()` would otherwise handle, since Vertex does not have one yet:

```
difference = guess - previous;
if (difference < 0) {
    difference = -difference;
}
```

It converges to full precision in five iterations for `sqrt(2)`, and lands exactly on `4.0` for the perfect square `sqrt(16)`, each result checked by squaring it back and asserting the error stays small. There is something worth sitting with here: this is a language built one deliberately small piece at a time, each piece justified only by the chapter that needed it, and it is already enough to run a centuries-old numerical method and verify its own answer. Worth reading end to end, not just for what it computes but for how little of the language it actually needs.

Considerations

1. Why must `continue` return to the condition rather than directly to the body?
2. Which changes to loop state establish progress toward termination?
3. Why does an inner loop consume `break` but propagate `exit`?

Practice

Answers to the practice questions follow this section.

1. Write a loop that sums the numbers from `1` through a value supplied by the caller. Add a `break` that stops early once the running total exceeds some bound.
2. Write a second version of the same sum that uses `continue` to skip multiples of a chosen number instead of adding them. In each version, say exactly where the counter update belongs and why moving it would change the result.
3. Predict the output of the nested-loop example in this chapter if the inner `break` condition were `inner == 3` instead of `inner == 2`, then run it and check.

Answers to the Practice Questions

1. Here the caller supplies `n=10` and `bound=12`. Check the bound after adding the current term:

   ```
   n = 10;
   bound = 12;
   total = 0;
   i = 1;
   while (i <= n) {
       total = total + i;
       if (total > bound) { break; };
       i = i + 1;
   };
   print(total);
   ```

This prints `15`: the term `5` is included before the bound check breaks the loop. The counter stays at `5` on that exit. Moving the update before the addition would skip the first term unless the initialization also changed. Checking the bound before adding would impose a different rule.

2. Advance the counter before a possible continue, so skipping a term still makes progress:

   Vertex has no remainder operator, so repeated subtraction checks divisibility:

   ```
   n = 10;
   multiple = 3;
   total = 0;
   i = 0;
   while (i < n) {
       i = i + 1;
       remainder = i;
       while (remainder >= multiple) {
           remainder = remainder - multiple;
       };
       if (remainder == 0) { continue; };
       total = total + i;
   };
   print(total);
   ```

   This prints `37`, the sum from 1 through 10 excluding 3, 6, and 9. The example assumes a positive whole-number `multiple`. An update after continue would be skipped precisely where progress is needed.

3. The nested-loop example accumulates into `combined`; it does not print inside either loop. Changing the condition to `inner==3` lets the inner body add twice per outer pass instead of once. Two outer passes each add `2+10`, so `combined` ends at `24` instead of `22`. Adding a semicolon after the final closing brace, followed by `print(combined);`, displays `24`.

References

1. Wikipedia contributors. “While loop.” Wikipedia. https://en.wikipedia.org/wiki/While_loop
2. Wikipedia contributors. “Halting problem.” Wikipedia. https://en.wikipedia.org/wiki/Halting_problem
3. Wikipedia contributors. “Loop invariant.” Wikipedia. https://en.wikipedia.org/wiki/Loop_invariant